DAA Project

Sunday, 19 October 2025 4:40 PM

PART 1

a) 1

```
Algorithm FindMinMax (A, low, high)
// input : Array A [low ... high] , output : (min, max)
 if low == high then
     return (A[low], A[low])
 else if high == low + 1 then
     if A[low] < A[high] then
         return (A[low], A[high])
     else
         return (A[high], A[low])
else
    mid ← (low + high) / 2
    (min1, max1) ← FindMinMax (A, low, mid)
    (min2, max2) ← FindMinMax (A, mid+1, high)
    return (min (min1, min2), max (max1, max2))
```

a) 2

Recurrence Relation: $T(n) = 2T(\frac{n}{2}) + 2$

↓ divide array into 2 halves ; ↓ compare 2 values

Solving with Substitution Method

$T(n) = 2T(\frac{n}{2}) + 2$

$T(\frac{n}{2}) = 2T(\frac{n}{4}) + 2$

substituting

$T(n) = 2[2T(\frac{n}{4}) + 2] + 2$

$= 4T(\frac{n}{4}) + 6$ —①

$T(\frac{n}{4}) = 2T(\frac{n}{8}) + 2$

substituting into ①

$T(n) = 4(2T(\frac{n}{8}) + 2) + 6$

$= 8T(\frac{n}{8}) + 14$

after k substitutions, pattern would be

$T(n) = 2^k T(\frac{n}{2^k}) + (2^{k+1} - 2)$

base case → $\frac{n}{2^k} = 1$

$\log_2^n = k$

hence, $T(n) = 2^{\log_2 n} T(\frac{n}{2^{\log_2 n}}) + (2^{\log_2 n + 1} - 2)$

$= nT(1) + 2n - 2$

$= O(n) + 2n - 2$

$= O(n)$

but we only count key comparisons so from basic derivation → $T(n) = 1.5n - 2$

$= O(n)$

a) 3

Brute force : $2(n-1)$ comparisons

Divide & Conquer : $1.5n - 2$ comparisons

a) 3

Brute force : 2(n-1) comparisons

Divide & Conquer : 1.5n-2 comparisons

Divide & Conquer is optimal in number of comparisons but slightly slower in practise due to recursion overhead

(25% less than brute force)

b) 1

```
Algorithm Power(a,n)
// Input: num a and positive int n
// output: a^n value
if n==0:
    return 1
temp = Power(a, n//2)
if n is even:
    return temp * temp
else:
    return a * temp * temp
```

b) 2

Recurrence Relation: $T(n) = T(\frac{n}{2}) + 1$

solving by master

$a=1$	$a = b^d$
$b=2$	$1 = 2^0$
$d=0$	$1 = 1$

hence $\longrightarrow$ $T(n) = O(n^0 \log n)$

$= O(\log n)$

b) 3

Brute force: n-1 multiplications

Divide & Conquer: $O(\log n)$ multiplications

Divide & Conquer is exponentially larger for large n

d) QUICK SORT COMPLEXITY

Worst case $\longrightarrow$ occurs when pivot is always the smallest or largest element

$T(n) = T(n-1) + O(n) = O(n^2)$

Best case $\longrightarrow$ occurs when the pivot always divides array into equal halves

$T(n) = 2T(\frac{n}{2}) + O(n) = O(n \log n)$